Module: Python Setup and First Programs

Previous module: Computer and Programming Foundations
Next module: Variables Datatypes Operators IO

Module Outcome:

- Run Python on your machine (REPL and a `.py` file) without guessing
- Print, comment, and save a tiny script you can show a classmate
- Know why Python is the language this course builds on

Why Python?

Python is a programming language created by Guido van Rossum (first released 1991). It reads almost like English, which helps learners learn faster than lower-level languages.

🏦 Bank 📦 Order 💻 Laptop 💰 Payroll

Where used?
- **SecureBank** / UPI scripts — small tools that print balances, receipts, or job status
- College apps — student portal banners, marks entry CLIs
- **TechMart** — canteen/order labels and first checkout demos
- **AcmeHR** — employee welcome scripts and HR training labs
- **MedCare** — appointment or ticket “hello” scripts before real CRUD

Why companies in India hire Python developers:

- **Web backends & APIs:** server-side apps and services (Instagram, Zomato-scale stacks often use Python)
- **Automation & scripting:** Reports, file processing, testing
- **Data & AI:** pandas, machine learning libraries
- **Full stack:** Python API + React frontend is a common startup stack

Python is **interpreted**: the computer runs your code line by line through the **Python interpreter**, without a separate compile step like C or Java (Java compiles to bytecode for JVM).

Installing Python 3

Download from https://www.python.org/downloads/ — choose **Python 3.12+** (not Python 2; it is obsolete).

Windows: Check **"Add python.exe to PATH"** during install.

Verify in terminal:

```
# macOS / Linux
python3 --version
# Example output: Python 3.12.4

# Windows
python --version
```

VS Code for Python

1. Install **Python** extension (Microsoft)
2. **Command Palette** (`Ctrl+Shift+P` / `Cmd+Shift+P`) → **Python: Select Interpreter** → choose Python 3.x
3. Open your course folder: `ksr_python_fullstack`
4. Create `projects/hello.py`

Running your first program

Two ways:

Mode	How	When to use
REPL (Read-Eval-Print Loop)	Type `python3` in terminal, see `>>>`	Quick experiments, calculator
Script	Save `.py` file, run `python3 hello.py`	Real projects, assignments

Keywords and identifiers

Keywords are reserved words Python uses for grammar — you cannot name a variable `if` or `class`.

Identifiers are names you choose: variables, functions, files (for variables).

Rules:

- Start with letter (a–z, A–Z) or underscore `_`
- Rest can be letters, digits, underscore
- **Case-sensitive:** `name` and `Name` are different
- No spaces: use `student_name` not `student name`
- Convention: **snake_case** for variables and functions in Python

Python vs Java vs C (brief)

Feature	C	Java	Python
Hello World lines	Many (includes boilerplate)	Class + main method	1 line
Semicolon	Required	Required	Not required
Blocks	`{ }` braces	`{ }` braces	Indentation
Type declaration	Explicit	Explicit	Dynamic (type decided at runtime)
Run	Compile then execute	Compile to bytecode, JVM	Direct interpret

C — Hello World (conceptual):

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Java — Hello World:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Python — Hello World:

```
print("Hello, World!")
```

For a **student management** CLI you will build over coming weeks, Python lets you focus on logic (add student, list students) instead of boilerplate.

Important Concepts and Definitions

Term	Definition
Python 3	Current major version of Python (use this, not Python 2)
Interpreter	Program that executes Python code line by line
REPL	Interactive shell: type code, see result immediately
Script	`.py` file saved and run as a whole
Statement	Instruction that does something (`print("Hi")`)
Keyword	Reserved word with special meaning to Python
Identifier	Name given to variable, function, class, module
snake_case	Naming style: `employee_id`, `total_marks`
Comment	Text ignored by Python — notes for humans (`#`)
PATH	OS list of folders where terminal finds `python` executable
Shebang	Optional first line `#!/usr/bin/env python3` on Unix scripts

Step-by-Step Notes (With Annotated Code)

Step 1: Environment checklist

Create `projects/env_check.py`:

```
# env_check.py — run once after install
import sys  # sys module gives Python version info (modules come later)

print("Python version:", sys.version)
print("Setup OK — ready for Core Python!")
```

Run:

```
cd ~/ksr_python_fullstack/projects
python3 env_check.py
```

Note:
- Wrong: Using Python 2 style (`print "hi"`) or a terminal where `python` is not found / PATH missing.
- Prefer: Install Python 3, use `python3` on Mac/Linux, and tick “Add to PATH” on Windows.
- Wrong: VS Code running a different interpreter than the one you installed.
- Prefer: Pick the correct Interpreter from the status bar.

Print the installed Python version and a ready line that includes Priya's name.

```
import sys

print(sys.version)
print("Ready for KSR Python — Priya")
```

Print the Python version and a ready line that includes Priya's name and the city Hyderabad.

```
import sys

city = "Hyderabad"
print(sys.version)
print(f"Ready for KSR Python — Priya ({city})")
```

Step 2: REPL — try before you save

```
python3
```

```
>>> print("Hello from REPL")
Hello from REPL
>>> 2 + 3
5
>>> exit()   # or Ctrl+D (macOS/Linux) / Ctrl+Z Enter (Windows)
```

REPL is great for testing one line; scripts are for programs you keep.

Print a greeting for Ravi and the sum of two TechMart canteen prices (₹450 and ₹120).

```
print("Hello, Ravi")
print(450 + 120)  # ₹570 for two canteen items
```

Expected Output:

```
Hello, Ravi
570
```

Print the learner name, the batch name, and a line that confirms Python 3 is OK.

```
print("Priya")
print("Full Stack Python Batch")
```

```
print("Python 3 OK")
```

Step 3: First script — hello world

`projects/hello.py`:

```
# hello.py
# Author: Your Name
# Purpose: First Python program for KSR Full Stack course

print("Hello, World!")
print("Welcome to Python Full Stack training.")
```

Run:

```
python3 hello.py
```

Annotation:

- `#` starts a **comment** — Python skips it
- `print(...)` is a **function** that displays text on screen
- Text in quotes is a **string literal**

Note:
- Wrong: Running without saving (tab still shows a dot) — you see old output; or `print(Hello)` without quotes.
- Prefer: Save (`Ctrl+S` / `Cmd+S`) before run; strings need `'...'` or `"..."`.

Print Priya's name and one-line career goal.

```
print("Priya")
print("Become a full-stack Python developer")
```

Expected Output:

```
Priya
Become a full-stack Python developer
```

Print the same name and career goal, and add author and date comments above the code.

```
# Author: Priya
# Date: 03-08-2026
print("Priya")
print("Become a full-stack Python developer")
```

Step 4: Multi-line output — training centre welcome

`projects/welcome_student.py`:

```
# Simulates welcome banner for student management system (future module)

print("=" * 40)          # repeat = character 40 times
```

```
print("   KSR STUDENT PORTAL")
print("=" * 40)
print("Login page loading...")
print("Please use registered email.")
print("=" * 40)
```

Output:

```
========================================
   KSR STUDENT PORTAL
========================================
Login page loading...
Please use registered email.
========================================
```

Print a SecureBank ATM welcome banner using repeated `=` lines around the title and welcome text.

```
print("=" * 24)
print("    SECUREBANK ATM")
print("=" * 24)
print("  Welcome, customer")
print("=" * 24)
```

Expected Output:

```
========================
    SECUREBANK ATM
========================
  Welcome, customer
========================
```

Print a student marks portal banner: title, a blank line, then a prompt to enter a roll number.

```
print("STUDENT MARKS PORTAL")
print()
print("Enter roll to continue")
```

Step 5: Keywords — what you cannot use as names

```
# INVALID — will cause SyntaxError
# if = 10        # 'if' is a keyword
# class = "A"    # 'class' is a keyword

# VALID identifiers
student_count = 45
employee_id = "EMP001"
_private_note = "internal use"   # leading _ convention: "internal"
```

Check keywords in REPL:

```
>>> import keyword
>>> print(keyword.kwlist)
# Long list including: False, None, True, and, as, assert, break, class, ...
```

Note:
- Wrong: Using a keyword as a variable name (`if = 10`) → `SyntaxError`.
- Prefer: Rename (`is_valid`, `if_flag`).

Print how many Python keywords this interpreter has, and show the first five names from `keyword.kwlist`.

```
import keyword

print(len(keyword.kwlist))
print(keyword.kwlist[:5])  # first five as a peek
```

Expected Output:

```
35
['False', 'None', 'True', 'and', 'as']
```

Remember: Keyword count can grow slightly across Python versions — always print `keyword.kwlist` on *your* machine in lab.

Keep invalid keyword assignments commented out, then print valid renamed variables (`if_flag`, `class_name`, `for_count`).

```
# INVALID (would cause SyntaxError if uncommented):
# if = 1
# class = "A"
# for = 5

# VALID renames:
if_flag = 1
class_name = "A"
for_count = 5
print(if_flag, class_name, for_count)
```

Step 6: Identifier naming — employee portal style

```
# Good names (clear, snake_case)
full_name = "Ananya Sharma"
department_code = "HR"
is_active = True

# Poor names (avoid in production code)
x = "Ananya Sharma"      # too vague
FullName = "Ananya"      # PascalCase reserved mostly for classes
employee id = "E1"       # SyntaxError — space not allowed
```

Note:
- Wrong: CamelCase everywhere for variables (`FullName`) — works but un-Pythonic.
- Prefer: `snake_case` for variables and functions.

Store a student name, roll number, and pass flag with clear `snake_case` names, then print each value.

```
student_name = "Priya"
roll_number = "2026001"
is_pass = True
print(student_name)
print(roll_number)
print(is_pass)
```

Expected Output:

```
Priya
2026001
True
```

Replace vague names like `x` / `nm` / `amt` with clear expense names and print the category with the amount in rupees.

```
# Poor: x, nm, amt  → clear snake_case
food_expense = 250
category_name = "food"
amount_inr = 250
print(f"{category_name}: ₹{amount_inr}")
```

Common Error: Putting a space in a name (`employee id = 101`) → `SyntaxError`. Use `employee_id`.

Step 7: Simple formatted greeting (preview of f-strings)

`projects/greet_trainer.py`:

```
trainer_name = "Kasi"
batch_name = "Python Full Stack July 2026"

# Comma print adds space between items automatically
print("Hello,", trainer_name)
print("Batch:", batch_name)

# f-string (preferred from Python 3.6+) — full detail in Module 02
print(f"Hello, {trainer_name}! You are teaching {batch_name}.")
```

Store Ravi's name and print a welcome message with an f-string.

```
name = "Ravi"
print(f"Hello, {name} — welcome to KSR")
```

Expected Output:

```
Hello, Ravi — welcome to KSR
```

Store a name and city, then print `{name} from {city}` using an f-string.

```
name = "Priya"
city = "Pune"
```

```
print(f"{name} from {city}")
```

Step 8: Script vs REPL workflow (recommended habit)

```
[Think] → [Try in REPL] → [Copy working lines to .py] → [Run script] → [Git commit later]
```

Example flow for expense tracker label:

```
# REPL experiment:
>>> print("Expense Tracker v0.1")

# Saved as projects/expense_banner.py:
print("Expense Tracker v0.1")
print("Module: Setup — no amounts yet")
```

Note:
- Wrong: Running a script from the wrong folder → `FileNotFoundError`.
- Prefer: `cd` to the folder that contains the script, or use a full path.

Print a one-line TechMart canteen bill title.

```
print("TechMart Canteen Bill")
```

Print the same TechMart canteen title, then a second line that says to pay at the counter.

```
print("TechMart Canteen Bill")
print("Pay at counter")
```

Practice Exercises

Basic

1. Install Python 3 and paste terminal output of `python3 --version` into `notes/setup_proof.txt`.

Model answer: After install, run:

`bash python3 --version`

Example output: `Python 3.12.x`. Create `notes/setup_proof.txt` and paste that exact line. On some Windows setups the command is `python --version` — either is fine if it shows 3.x.

2. Write `hello.py` that prints your name and city on two lines.

Model answer:

```
print("Priya Sharma")
print("Hyderabad")
```

Run: `python3 hello.py`

3. In REPL, print the result of `10 * 5` and `100 / 4`.

Model answer: Start REPL with `python3`, then:

```python

    10 * 5
    50
    100 / 4
    25.0
    ```
```

Note: `/` always returns a float in Python 3.

4. List 10 Python keywords from `keyword.kwlist`.

**Model answer:**

```
import keyword
print(keyword.kwlist[:10])
```

Example first ten: `False`, `None`, `True`, `and`, `as`, `assert`, `async`, `await`, `break`, `class` (exact order can vary slightly by version — print the list on your machine).

Intermediate

1. Write `training_banner.py` that prints a 5-line banner for "Employee Portal Training".

Model answer:

```
print("==============================")
print("  Employee Portal Training")
print("  KSR Full Stack Python")
print("  Welcome, learners!")
print("==============================")
```

2. Fix this code (find 3 errors):

`python Print("Hello") class = "Python" employee id = 101`

Model answer: Errors: (1) `Print` must be `print` (case-sensitive). (2) `class` is a keyword — cannot be an identifier. (3) `employee id` has a space — use underscore.

`python print("Hello") course = "Python" employee_id = 101`

3. Create identifiers for: student roll number, monthly salary, is_logged_in flag.

Model answer: Prefer `snake_case`: `student_roll_number` (or `roll_no`), `monthly_salary`, `is_logged_in`. Avoid spaces, starting with a digit, or keywords.

4. Write same greeting in REPL and in script; note differences in workflow.

Model answer:
- **REPL:** type `print("Namaste")` → see result immediately; good for experiments; lost when you quit unless you save history.
```

- **Script:** save `greeting.py` with the same `print`, run `python3 greeting.py` anytime; reusable and shareable. Industry work almost always uses scripts/files, not only REPL.

Interview-Oriented

1. Is Python compiled or interpreted? Explain in 2–3 sentences for an interviewer.

Model answer: Python is primarily an **interpreted** language: the CPython interpreter reads source (or bytecode) and executes it. Source is compiled to bytecode (`.pyc`) behind the scenes, then run on the Python Virtual Machine — so it is not "compiled to machine code like C," but also not purely line-by-line with no bytecode. In interviews, "interpreted / high-level" is the expected short answer, with bytecode as a bonus detail.

2. Why is `snake_case` preferred over `camelCase` in Python community?

Model answer: PEP 8 (Python style guide) recommends `snake_case` for functions and variables. Consistent style makes code readable across teams and matches the standard library. `camelCase` is common in Java/JavaScript; mixing styles in Python looks non-idiomatic in reviews.

3. Compare one advantage and one limitation of Python vs Java for enterprise backends.

Model answer: Advantage: Python usually has faster development and clearer syntax for APIs, scripting, and data work. **Limitation:** raw CPU-heavy workloads and some large enterprise ecosystems still lean on Java's performance, strict typing culture, and long-standing tooling — Python can be slower for tight loops unless you use libraries (NumPy) or other services.

4. What happens if you name a variable `True`? (Try in REPL.)

Model answer: `True` is a keyword / built-in constant. Assignment like `True = 1` raises `SyntaxError: cannot assign to True`. You cannot use keywords as variable names — choose `is_true` or `flag` instead.

Mini-Project (Assignment): Environment & Greeting Pack

Title: Prove your workspace + first greeting scripts
Lab kit (build here): 15_Lab_Assets/27_terminal_workspace/

What to build

1. `projects/env_check.py` — prints Python version
2. `projects/hello.py` — hello world
3. `projects/my_intro.py` — prints your name, career goal, and one reason you chose Python full stack
4. `notes/python_vs_java.md` — table comparing 5 syntax features (from this module)
5. Screenshot: VS Code running `my_intro.py` with output in terminal

Stretch

- Add a shebang line and make `my_intro.py` executable on macOS/Linux (`chmod +x my_intro.py`).
- Add `projects/welcome_batch.py` that prints a 5-line KSR batch banner.

Done when

- [ ] Lab kit ticket folders/scripts prove a clean workspace
- [ ] `env_check.py`, `hello.py`, and `my_intro.py` all run without errors
- [ ] Comparison notes file exists with at least 5 rows

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why did Python become popular for beginners but also for companies like Instagram?
2. If indentation replaces braces, what could go wrong when copy-pasting code from the web?
3. what feels easier in Python compared to anything you tried before?
4. Should every `.py` file start with `import` statements? When is bare `print` enough?
5. How is selecting "Python interpreter" in VS Code similar to choosing which translator reads your book?

Q1. What is Python? What is it used for?

A: High-level, interpreted language known for readability; used in web backends, APIs, automation, data science, and scripting.

Q2. Difference between interpreted and compiled languages?

A: Interpreted (Python) runs via interpreter line-by-line; compiled (C) translates to machine code first. Java is hybrid: compile to bytecode, run on JVM.

Q3. What are Python keywords? Example of invalid use?

A: Reserved words like `if`, `def`, `class`; cannot use as variable names — `class = 5` is invalid.

Q4. How do you run a Python file from command line?

A: `python3 filename.py` (or `python filename.py` on Windows) from directory containing file or with full path.

Q5. Why is Python popular for startups in India?

A: Fast development, rich libraries, easier hiring/training, and a strong fit for MVPs, automation, and APIs.
